PSM: Timely and Resource-Efficient Sketch Migration in Network Measurement

Xun Zhou *, Hongyan Liu †, Xiang Chen ‡, Zhengyan Zhou ‡, Zihao Cheng *,
Di Wang *, and Chunming Wu ‡
*College of Control Science and Engineering, Zhejiang University, China
†College of Computer and Data Science, Fuzhou University, China
‡College of Computer Science and Technology, Zhejiang University, China

2026 IEEE/ACM International Symposium on Quality of Service (IWQoS) | 979-8-3195-1703-6/26/$31.00 ©2026 IEEE | DOI: 10.1109/IWQoS70441.2026.11661141

***Abstract*—Sketches can achieve resource-efficient and high-accurate network measurement (NM) with configurable resource-performance trade-offs. Due to network dynamics, network measurement points (i.e., the switches deployed with sketches) need to reallocate dynamically to cover full traffic. However, using existing approaches to migrate sketches leads to unacceptable latency and resource overhead. In this paper, we propose PSM, a framework for timely and resource-efficient sketch migration. For timeliness, PSM utilizes a protocol that is able to access sketch counters and migrate them in parallel using background traffic. To ensure migration without direct traffic, we leverage programmable switches' mirroring and recirculation functions to generate packets. For resource efficiency, PSM uses bitmap structures and hardware-friendly computations to track the migration progress and verify integrity. We implement PSM on Intel Tofino2 switches and compare PSM with three state-of-the-art approaches through extensive experiments. Results indicate that PSM achieves timely and resource-efficient sketch migration.**

I. Introduction

Based on configurable resource-performance trade-offs, sketches accomplish network measurement (NM) tasks with low resource overhead and limited error bounds [1]. The development of programmable network further enhances the practicality of sketches [2]–[4]. By dynamically configuring flow table rules, sketches can be integrated into the data plane using a structure of parallel counters, empowering NM applications (e.g., heavy hitter detection and global iceberg detection) [5]–[7]. Benefit from network programmability, sketch-based NM is demonstrated as promising solutions [8].

Due to network dynamics, the original network measurement points (NMPs) (i.e., switches deployed with sketches) fail to cover all traffic and must reallocate dynamically to ensure traffic coverage. For example, ECMP is a load balancing protocol that changes traffic distribution [9]. In this dynamic situation, the source NMPs need to piggyback sketch counters on the packet headers and deliver them to the destination NMPs, so as to maintain the stateful sketch information.

However, to the extent of our knowledge, state-of-the-art approaches utilize random migration, serial migration, and even control plane migration to migrate sketches between the source and destination NMPs [10]–[12]. They fall short of achieving timeliness and resource efficiency due to the following two major limitations. (1) Inefficient access to sketch counters. Sketches efficiently store the NM results by maintaining parallel counter arrays [13]. However, the existing serial migration method [11] accesses the counters of each array one by one in sequence, resulting in untimeliness. (2) High switch resource consumption. Programmable switches have limited storage and computing resources, but existing method of siphash and RSA in [11] consumes nearly half of the switch computing resources for integrity. Even though control plane migration in [12] moves some of the complex calculations to servers, it introduces high communication overhead between the control plane and the data plane [14].

979-8-3195-1703-6/26/$31.00 ©2026 IEEE

To overcome the above limitations, we propose PSM, a data plane framework for timely and resource-efficient sketch migration. The key idea is to leverage the header modifying capability of programmable switches to enable sketch migration between source and destination NMPs. Further, PSM employs three major designs to enhance timeliness and resource efficiency. (1) The PSM protocol fits the structure of sketch counter arrays and is able to piggyback multiple counters with the same position in counter arrays together in parallel, which enhances timeliness. (2) PSM provides two optional migration strategies for different network scenarios (i.e., the Background Flow (BF) strategy and the Mirroring and Recirculation (MR) strategy). When there is stable traffic between NMPs, PSM migrates sketches through background traffic (i.e., normal traffic). Otherwise, PSM utilizes the mirroring and recirculation functions of programmable switches to generate packets by the switch itself. This design enhances the universality and reliability of sketch migration. (3) PSM utilizes bitmap structures and hardware-friendly computations to track the migration process and verify migration integrity, achieving resource efficiency. The source code of PSM is released on GitHub for open access [15].

In summary, this paper makes the following contributions:

- We classify three scenarios of sketch migration and analyze the limitations of existing methods (§II).
- We propose PSM, a data plane framework for timely and resource-efficient sketch migration, including the PSM protocol, the BF and MR migration strategies, and the integrity verification strategy (§III).
- We implement PSM on Intel Tofino2 switches and compare it with state-of-the-art approaches. Results indicate timely and resource-efficient sketch migration (§IV).

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:34:29 UTC from IEEE Xplore. Restrictions apply.

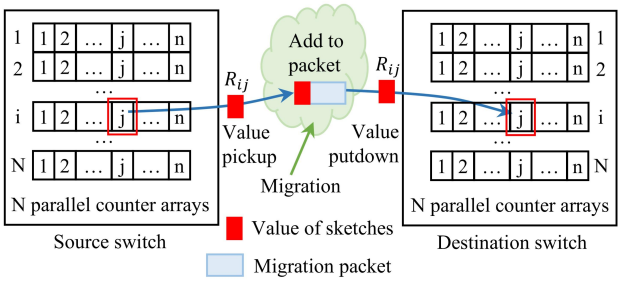


Fig. 1. Sketch migration process

II. Background and Motivation

A. Sketch Migration Situations

In practical NM scenarios, there are three major cases where sketches need to be migrated between NMPs.

Sketch load balancing: Sketch load balancing happens when plenty of flows flood into certain NMPs, causing sketch counter overload [16]. In this situation, sketches need to be migrated and stored in other distributed NMPs to realize sketch load balancing and avoid possible counter overflow.

NM intent changes: Different NM intents require completion at different NMPs [17]. When the NM intent changes, the new NMPs need to synchronize with the original measurement data (i.e., sketches) to prevent incorrect NM results. One may issue that new NMPs can start a new sketch for NM. However, sketches are stateful, and their accumulated data in counters is critical for accurate detection and cannot be discarded arbitrarily. Take heavy hitter detection as an example [5], at the source NMP, a sketch tracks per-flow counts. A flow exceeding a threshold (e.g., 10,000 packets) is flagged as a heavy hitter. If a flow is identified as a heavy hitter at the source NMP, but the destination NMP's sketch resets, this flow bypasses detection until it re-reaches the threshold, which significantly affects the overall NM results.

Link failure: When a network link fails, traffic on the original paths will be rerouted [18]. This may make the flows that should have passed through the original NMPs need to be measured by others [16]. To ensure NM accuracy, after the network routing has been re-planned, we need to migrate sketches from the source NMPs to the destination NMPs, which can cover all traffic under the new routing scheme.

B. Problem Statement

The goal of sketch migration is to piggyback sketch counters on packet headers and deliver packets from source NMPs to destination NMPs, as shown in Fig 1. In programmable switches, sketches are stored in a set of parallel counter arrays $R_1, \ldots, R_N$. Each array R_i, $i \in [1, N]$, has the same length n (i.e., n sketch counters). And each position of the counter arrays can be indexed by an integer j, $j \in [1, n]$, where R_{ij} is the j-th position of counter array R_i. Sketch migration is a periodic event, and we migrate all the sketch counters of all the counter arrays one by one, i.e., $\forall i \in [1, N]$, $j \in [1, n]$, we migrate R_{ij} from the source NMPs to the destination NMPs.

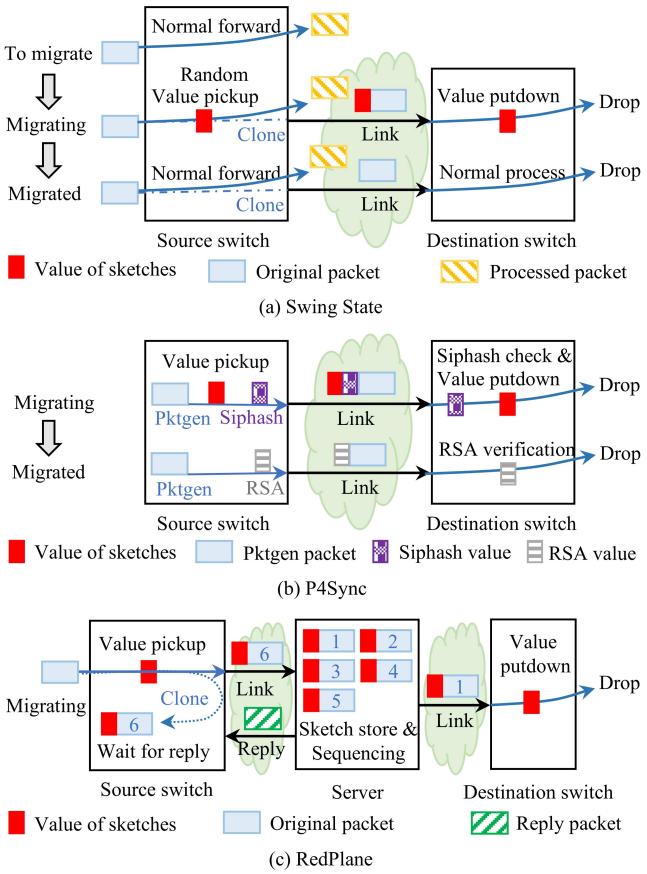


Fig. 2. Workflow of start-of-the-art approaches

C. Limitations of Existing Solutions

Up to now, there have been three state-of-the-art approaches for sketch migration. Their workflows are shown in Fig 2.

Swing State [10]. Swing State is the first to propose the migration task in SDN. As shown in Fig 2(a), when a packet enters, it clones a copy. The original packet is sent to its original destination, and the mirrored packet randomly carries sketch counters in a specially designed protocol and is then forwarded to the destination NMPs. However, random migration causes some sketch counters to be repeatedly migrated, while others may have never been migrated. The repetition and incompleteness interfere with timeliness and integrity.

P4Sync [11]. P4Sync optimizes the Swing State design to realize a sequential rather than random migration. As shown in Fig 2(b), it further achieves integrity verification through siphash and RSA encryption. This is designed to deal with packet loss during migration, but introduces complex computations. In addition, limited by the design of public registers, P4Sync can access only one sketch counter at a time [11]. Considering the parallel structure of counter arrays, we can actually carry multiple counters together.

RedPlane [12]. RedPlane is an advanced control plane method that utilizes an intermediate server to temporarily store sketches. As shown in Fig 2(c), on receiving the sketches from the source NMPs, the server conducts a sequential verification. If the expected sequence number is not received and a timeout

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:34:29 UTC from IEEE Xplore. Restrictions apply.

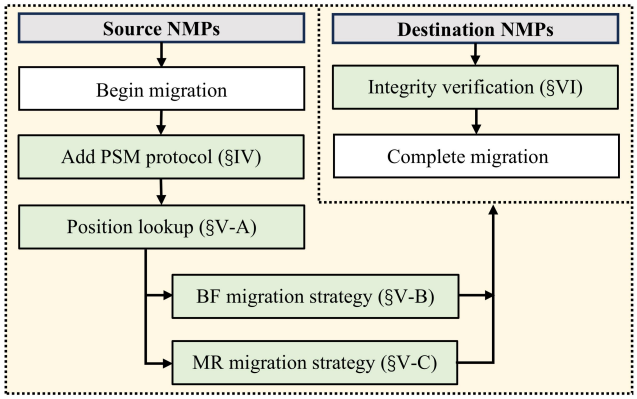


Fig. 3. PSM framework

occurs, secondary migration is required. After the source NMPs finish migration, the server starts to forward packets to the destination NMPs. Due to control plane usage, RedPlane introduces additional latency for communication between the control plane and the data plane, which is basically hundreds of milliseconds [14], interfering with timeliness.

III. PSM Design

A. Design Overview

Collectively, we adopt three core designs in PSM, as shown in Fig 3. First, we design the PSM migration protocol that is able to access and carry multiple counters with the same position in the counter arrays together, overcoming the hardware limitations and achieving parallel migration (§III-B). This design reduces the packet usage during migration and thus enhances timeliness. Second, PSM uses bitmap structures to track the migration process in the source NMPs (§III-C) and store the indexes of updated sketch counter positions in the destination NMPs for integrity verification (§III-F). This bitmap design occupies limited storage and supports hardware-friendly computations. Third, PSM provides two optional strategies, BF and MR. In particular, the BF strategy employs the background traffic (i.e., normal traffic) to carry sketches (§III-D), and the MR strategy utilizes the mirroring and recirculation functions of the programmable switches to generate packets by the switch itself (§III-E).

B. PSM Migration Protocol

We introduce the PSM protocol that fits the parallel structure of counter arrays. As shown in Fig 4. For sketches with N counter arrays, the PSM migration protocol contains $N + 3$ fields. The *proto_id* field is used to connect the two protocols. Subsequently, the *read_place_id* field records the index of the current migration position of counter arrays, so as to track the migration process. In the destination NMPs, this field is used to find the related position of migrated counters. The *N load_sketch* fields store the values of all the sketch counters with the same position in the counter arrays. Finally, the *if_finish* field is set to one when this is the last packet for migration, prompting the destination NMPs to start the integrity verification process.

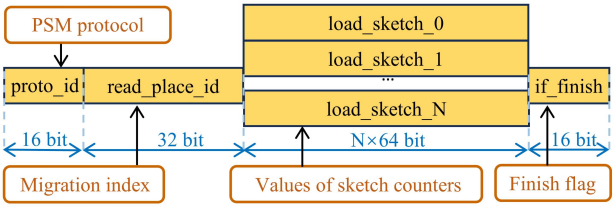


Fig. 4. Structure of the PSM migration protocol

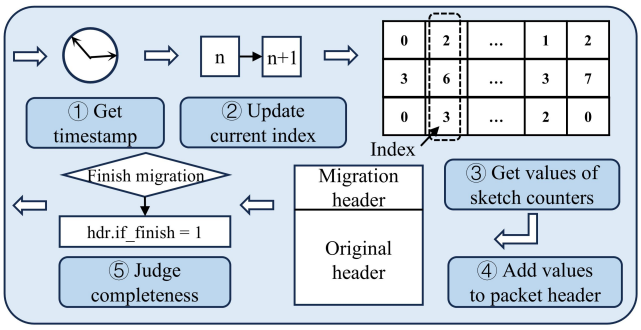


Fig. 5. Workflow of the BF migration strategy

C. Migration Position Lookup Strategy

PSM migrates the counters in the parallel sketch counter arrays column by column, as shown in the third step of Fig 5. This is achieved by maintaining a bitmap structure with only one counter. When a packet enters the source NMPs, the value of this bitmap structure is accessed and incremented by one, which represents the current migration index of the counters. Then, we access all the parallel counter arrays using this index. PSM repeats this process until the index equals the array length, which means that sketch migration finishes.

D. BF Migration Strategy

When a normal packet enters the source NMPs, we first obtain the internal timestamp from the switches for calculating the migration time later. Accordingly, the packet is also timestamped when it leaves the destination NMPs. Next, we make a copy of this packet within the source NMPs. The original packet is forwarded in its original manner, and the copied packet proceeds with the subsequent process, which is shown in Fig 5. We track the migration process through the aforementioned bitmap structure and access sketch counters in parallel. The values of these counters are added to the packet header where the PSM protocol is placed. Before forwarding the copied packet, we check if the migration is finished. If true, the *if_finish* field of the PSM protocol is set to one, which will enable integrity verification in the destination NMPs.

E. MR Migration Strategy

When there is no background traffic between the source and destination NMPs, existing methods can not work [10]–[12]. By leveraging the ability of programmable switches to autonomously generate packets through mirroring and recirculation, PSM can achieve sketch migration without background traffic and thus enhance migration reliability. Fig 6 shows the workflow of the MR migration strategy. A control packet

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:34:29 UTC from IEEE Xplore. Restrictions apply.

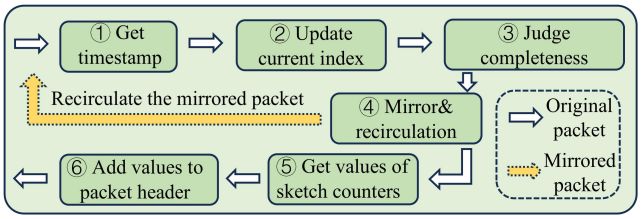


Fig. 6. Workflow of the MR migration strategy

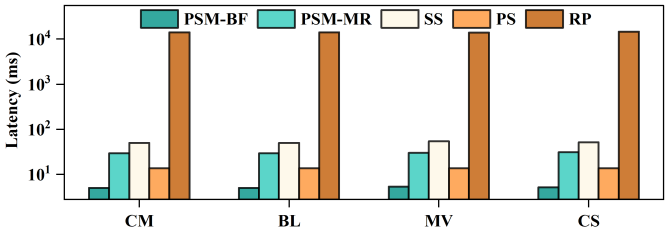


Fig. 7. (Exp#1) Migration latency of four different sketches

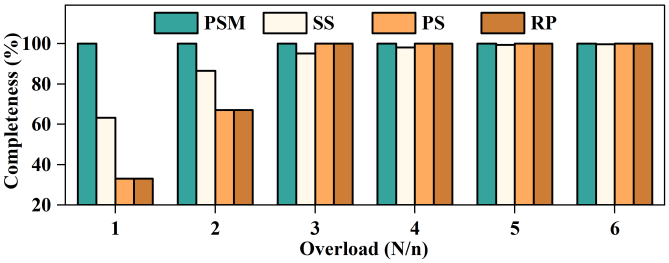


Fig. 8. (Exp#2) The impact of packet overhead on migration completeness

is used to notify the start of migration. When it enters the source NMPs, it gets a timestamp and a migration index. This is similar to the BF strategy. After that, we first judge migration completeness to avoid redundant mirroring. If sketch migration is not finished, this packet is mirrored by the switch. The original packet is forwarded to the egress of the switch pipeline, and will access the sketch counters there. This egress setting is due to the processing capabilities of the Intel Tofino2 switches [19]. The mirrored packet is forwarded to a certain port of the switch for recirculation. In our experiment, a packet sent to port 0 is recirculated, and it will be captured at port 128 in the switch ingress. Through incoming port filtering, we can distinguish the mirrored packet from normal packets. This packet repeats this workflow until migration finishes.

F. PSM Integrity Verification

During migration, when a packet enters the destination NMPs, it updates the sketch counters together with the bitmap structure (i.e., the verification bitmap) to record the index of updated counters. After migration, a packet is recirculated inside the switch for integrity verification, so as to cope with the restriction of counter access in Intel Tofino2 switches [19]. In particular, for each round of recirculation, this packet can check one counter in the bitmap structure. If a counter is not updated, its index is recorded and added to the packet header. After checking all the counters of the bitmap, this packet is sent to the source NMPs for secondary migration. Through this integrity verification design, we ensure the migration reliability and resource efficiency of PSM.

A strawman solution is to use the gRPC function of the control plane to record the migration process and verify integrity. However, communications between the control plane and the data plane require a latency of hundreds of milliseconds [14], which highly interferes with the timeliness requirement of NM. Therefore, we design the PSM integrity verification strategy entirely in the data plane.

IV. Evaluation

A. Experiment Settings

Testbed settings. Our testbed has two 32 × 100Gbps Intel Tofino2 switches and two servers with 36-core Intel(R) Xeon(R) Gold 6240C CPU(2.60GHz) and 128 GB RAM. The two switches are used as the source and the destination NMPs. The migration process is carried out between them. We use DPDK-Pktgen 25.03.0 [20] at the server on the source switch side to send packets as background traffic. We capture packets and parse experimental data at the other server. All the devices are connected via a 10 Gbps link in the network.

As for sketch utilization, we choose four commonly used sketches, namely Count-Min Sketch (CM) [21], Bloom Filter (BL) [22], MV Sketch (MV) [23], and Count Sketch (CS) [24]. We configure these sketches with respect to the settings in [25]. The adjustable parameter N of the PSM protocol is set to three to match the configuration of the four sketches and better compare the experimental results.

B. Evaluation Results

(Exp#1) Migration latency. We measure the latency of the complete migration process on four different sketches. For all the migration methods, when a packet enters the source switch ingress and leaves the destination switch egress, it gets a nanosecond precision timestamp, respectively. At the packet-receiving server, packets are parsed to get these two timestamps, through which we can calculate the overall sketch migration latency, as mentioned in (§III-D).

As shown in Fig 7, the BF strategy of PSM has the shortest latency, and it takes only 5ms to complete sketch migration. The MR strategy of PSM involves packet mirroring and recirculation, which incurs additional link time, but the overall migration delay is maintained at 30ms. Considering situations with no background traffic between NMPs, where BF-based methods (e.g., PS) can not work, this limited delay increase is cost-effective. Due to random migration, SS needs to take 11 times the packets of the counter array length to complete migration on average [11]. PS takes three times the time of the BF strategy, due to serial sketch counter access. With control plane usage, RP reaches second-level latency. Experimental results indicate that PSM has the least migration latency and achieves timely sketch migration.

(Exp#2) Overhead of migration packets. We measure the number of packets used for migration, which is a key factor that reflects latency and bandwidth occupancy. If the N counter arrays each has n counters, and we use N_p packets

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:34:29 UTC from IEEE Xplore. Restrictions apply.

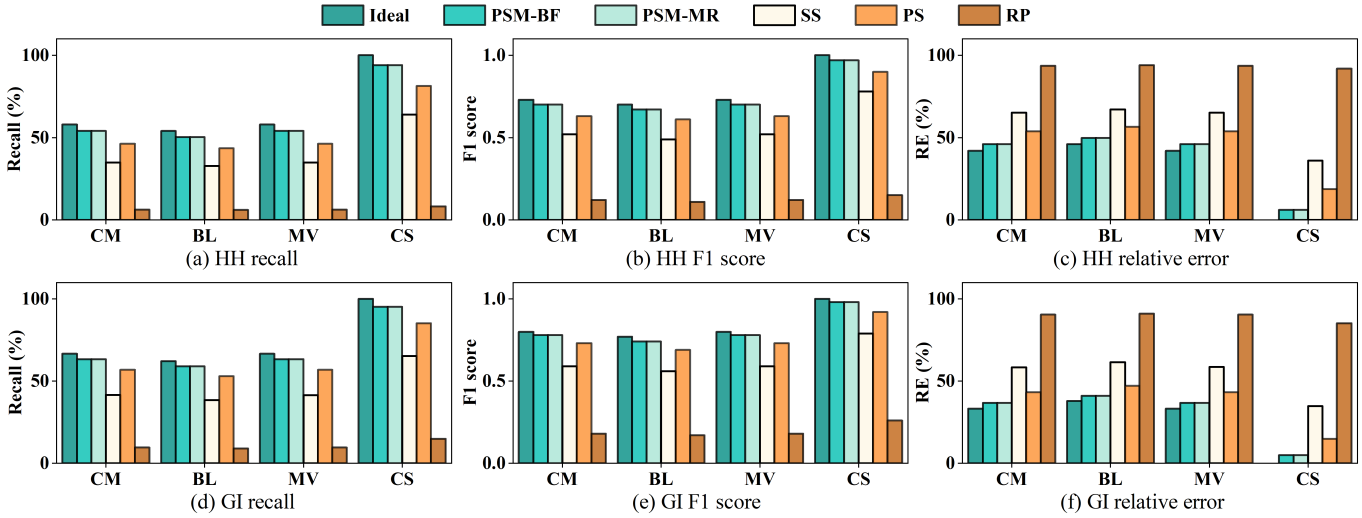


Fig. 9. (Exp#3) The sketch migration effect on two sketch-based NM applications

to complete sketch migration, then the overhead of packets is defined as N_p/n. As for the random migration of SS, it is hard to determine the exact migration completion time in programmable switches. Thus, we follow the theoretical analysis result in [11]. Moreover, since the BF strategy and the MR strategy have the same packet overload during migration, we put their results together as "PSM".

Fig 8 illustrates the change in migration completeness as the number of packets increases. PSM achieves the most efficient migration. For $N = 3$ in our experiments, PS and RP use three times the packets of PSM. Due to random migration, SS only achieves 63% completeness when overhead is one and 95% when overhead is three. However, it needs more than ten times the packets of PSM to achieve complete migration. The experimental results show the design efficiency of the PSM.

(Exp#3) Effect on sketch-based NM applications. We test the sketch migration methods on two real-world sketch-based NM applications (i.e., heavy hitter detection (HH) and global iceberg detection (GI)) [5], [6]. For each application, recall, F1 score and relative error (RE) are measured [25]. We also utilize the original sketches without migration to conduct the same experiment as baseline "Ideal". We utilize four sketches and configure this experiment with respect to the settings in [25] for fair comparison.

As shown in Fig 9, the sketch migration process inevitably reduces the effect of these two NM applications due to migration latency and incompleteness. Specifically, long latency leads to untimely sketch updates, and incompleteness results in missing sketch contents. This effect reduction is most evident in RP because of significantly untimely sketch updates in the destination switch. Compared with the state-of-the-art sketch migration methods, PSM increases recall and F1 score by 8% and 5%, and reduces relative error by 8%. Experimental results indicate that PSM achieves timely and reliable sketch migration, enhancing the effect of NM applications.

(Exp#4) Switch hardware resource utilization. We measure the occupancy of switch hardware resources, including Stage, SRAM, ALU and VLIW. We choose these four key metrics, because they can respectively reflect pipeline computing resources, storage resources, logical computing resources, and parallel processing resources of programmable switches [25]. After the migration programs of the four sketches are compiled and deployed on the switches, we can get the resource usage through compilation logs. We also conduct the same experiment using programs with only sketches as baseline. The average hardware resource usage is shown in Table I.

TABLE I
(Exp#4) The average resource usage of the sketch migration methods

Method	Stage	SRAM (%)	ALU (%)	VLIW (%)
Only Sketch	3.4	0.73	6.25	2.08
SS	5	8.02	10.42	3.28
PS	9	9.78	14.95	5.06
RP	8	9.04	12.50	4.53
PSM-BF	4.6	**7.70**	10.00	3.45
PSM-MR	**4**	7.81	**8.33**	**2.50**

(1) Stage. There are 12 stages in the programmable switches [26]. Compared with programs with only sketches, PSM and SS only bring an additional occupancy of 1.2 to 1.6 stages. However, PS and RP occupy additional 4.6 to 5.6 stages, reaching nearly half of the stage resources, which may interfere with deploying other network functions. (2) SRAM. The SRAM usage of all the methods is similar, but PSM still reduces 0.21%, 1.97%, and 1.23% SRAM, compared with three state-of-the-art methods. (3) ALU. The MR strategy of PSM has the least ALU usage of 8.33%, and PS occupies the most ALU of 14.95%. Comparing with programs that only contain sketches, PSM additionally occupies only 3.75% ALU usage. (4) VLIW. The MR strategy of PSM occupies only 2.5% VLIW on average. Although that of the BF strategy is slightly higher than SS, PSM is still more resource-efficient than the other two methods. In summary, considering the additional switch resource usage caused by sketch migration,

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:34:29 UTC from IEEE Xplore. Restrictions apply.

TABLE II
(Exp#5) The efficiency of integrity verification

Method	Latency	Packet usage	Stage	VLIW
PSM	340.0 ns	1	2	4.69%
PS	440.8 ns	$3n$	2	6.25%
Optimization	100.8 ns	$3n$-1	0	1.56%

PSM reduces 25% stage, 3% SRAM and 10% ALU. The VLIW usage of PSM is also limited.

(Exp#5) Integrity verification efficiency. We measure the efficiency of the integrity verification. Considering the long waiting time of RP, PSM only compares with PS. We conduct the experiment based on CM. By adding timestamps at the ingress and the egress of the destination switch, we can get the integrity verification latency caused by the recirculation process in PSM, which shows the computational complexity of verification design. The latency of siphash in PS can be calculated in the same way. As for switch resource usage, we choose stage and VLIW, because they are highly related to counter access and sketch content validation [25].

As shown in Table II, PSM only spends an average of 340ns for verifying one counter. This is achieved by recirculating only one packet inside the destination switch, which is more efficient than PS. Moreover, although the two strategies have the same stage usage, PSM reduces the VLIW usage by 1.56%. Results indicate that PSM achieves migration integrity with timeliness and resource efficiency.

V. Conclusion

This paper introduces PSM, a framework for timely and resource-efficient sketch migration in NM. PSM piggybacks sketches on packet headers in parallel via the PSM protocol and leverages BF and MR strategies to migrate stateful sketch contents between source-destination NMPs. PSM also employs bitmap structures to track the migration process and verify integrity. We have implemented the PSM framework on Intel Tofino2 switches. Extensive experiments indicate that PSM achieves timely and resource-efficient sketch migration.

Acknowledgments

This work was supported by the Science and Technology Program of Fujian Province under Grant No.2025H6007 and the National Key R&D Program of China under Grant No. 2023YFB2904000. Corresponding author: Hongyan Liu (hyliu20@zju.edu.cn).

References

[1] Z. Zhou *et al.*, "SketchGuide: Reconfiguring sketch-based measurement on programmable switches," in *2022 IEEE 30th International Conference on Network Protocols (ICNP)*, 2022, pp. 1–11.

[2] H. Zheng *et al.*, "Rethinking fine-grained measurement from software-defined perspective: A survey," *IEEE Transactions on Services Computing*, vol. 15, no. 6, pp. 3649–3667, 2022.

[3] X. Chen *et al.*, "Eagle: Toward scalable and near-optimal network-wide sketch deployment in network measurement," in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24. Association for Computing Machinery, 2024, p. 291–310.

[4] Y. He and X. Zhang, "A survey on network measurement for software-defined networks," in *2019 3rd International Conference on Electronic Information Technology and Computer Engineering (EITCE)*, 2019, pp. 1534–1540.

[5] Q. Xiao, Z. Tang, and S. Chen, "Universal online sketch for tracking heavy hitters and estimating moments of data streams," in *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications*, 2020, pp. 974–983.

[6] H. Zhao, A. Lall, M. Ogihara, and J. Xu, "Global iceberg detection over distributed data streams," in *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*, 2010, pp. 557–568.

[7] X. Zhou *et al.*, "LC-Sketch: A layered-carry sketch for IoT network measurement," in *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026, pp. 19 952–19 956.

[8] X. Chen *et al.*, "Phantom: Virtualizing switch register resources for accurate sketch-based network measurement," in *Proceedings of the Twentieth European Conference on Computer Systems*, ser. EuroSys '25. Association for Computing Machinery, 2025, p. 1383–1398.

[9] J.-L. Ye *et al.*, "A weighted ECMP load balancing scheme for data centers using P4 switches," in *2018 IEEE 7th International Conference on Cloud Networking (CloudNet)*, 2018, pp. 1–4.

[10] S. Luo, H. Yu, and L. Vanbever, "Swing State: Consistent updates for stateful and programmable data planes," in *Proceedings of the Symposium on SDN Research*, ser. SOSR '17. Association for Computing Machinery, 2017, p. 115–121.

[11] J. Xing, A. Chen, and T. S. E. Ng, "Secure state migration in the data plane," in *Proceedings of the Workshop on Secure Programmable Network Infrastructure*, ser. SPIN '20. Association for Computing Machinery, 2020, p. 28–34.

[12] D. Kim, J. Nelson, D. R. K. Ports, V. Sekar, and S. Seshan, "RedPlane: enabling fault-tolerant stateful in-switch applications," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, ser. SIGCOMM '21. Association for Computing Machinery, 2021, p. 223–244.

[13] H. Liu *et al.*, "Elastically scaling control channels in network measurement with Escala," vol. 33, no. 2, pp. 777–792, 2025.

[14] X. Chen *et al.*, "ApproSync: Approximate state synchronization for programmable networks," in *2020 IEEE 28th International Conference on Network Protocols (ICNP)*, 2020, pp. 1–12.

[15] PSM source code. (2026). [Online]. Available: https://github.com/zxzx2002/Sketch_Migration.git

[16] C. Sun *et al.*, "OFM: Optimized flow migration for NFV elasticity control," in *2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS)*, 2018, pp. 1–10.

[17] B. E. Ujcich, A. Bates, and W. H. Sanders, "Provenance for intent-based networking," in *2020 6th IEEE Conference on Network Softwarization (NetSoft)*, 2020, pp. 195–199.

[18] T. Chang and L. Cai, "Multipath routing compatible congestion control," in *2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall)*, 2024, pp. 1–5.

[19] H. Liu *et al.*, "Toward resource-efficient and high-performance program deployment in programmable networks," *IEEE/ACM Transactions on Networking*, vol. 32, no. 5, pp. 4270–4285, 2024.

[20] DPDK. (2026). [Online]. Available: https://www.dpdk.org/

[21] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.

[22] Q. He, Z. Li, C. Chen, and H. F. and, "Research on global bloomfilter-based data routing strategy of deduplication in cloud environment," *IETE Journal of Research*, vol. 70, no. 3, pp. 2705–2715, 2024.

[23] L. Tang, Q. Huang, and P. P. C. Lee, "MV-Sketch: A fast and compact invertible sketch for heavy flow detection in network data streams," in *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*, 2019, pp. 2026–2034.

[24] M. Charikar, K. Chen, and M. Farach-Colton, "Finding frequent items in data streams," *Theoretical Computer Science*, vol. 312, no. 1, pp. 3–15, 2004.

[25] X. Chen *et al.*, "MTP: Avoiding control plane overload with measurement task placement," in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*, 2021, pp. 1–10.

[26] Intel Tofino. (2026). [Online]. Available: https://www.intel.com/content/www/us/en/products/details/network-io/intelligent-fabric-processors/tofino.html

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:34:29 UTC from IEEE Xplore. Restrictions apply.